

Perform a Cross-Site Request Forgery (CSRF) Attack

Execution mode	Authorized, isolated training lab; no public systems were targeted.
Output status	Representative/sanitized lab output recorded for submission. Replace with instructor-required screenshots if a live lab run is performed.

Aim

Demonstrate in a local DVWA lab how a missing anti-CSRF token can allow an unwanted state-changing request.

Theory

CSRF abuses the browser trust relationship created by an authenticated session. The test below uses a local account and a same-machine training page. It does not target a real victim or send a link to anyone.

Tools and Lab Scope

- DVWA low-security lab
- Burp Suite
- Text editor
- Local Python HTTP server
- Browser test profile

Procedure and Recorded Execution

Step 1. Configure the lab - Log in to the local DVWA account and select the low training security level.

```
target: http://192.168.56.30/dvwa
account: lab-admin (disposable)
```

Step 2. Inspect the state-changing request - Confirm that the password-change request has no per-request CSRF token.

```
GET /dvwa/vulnerabilities/csrf/?password_new=...&password_conf=...&Change=Change
csrf token: absent
```

Step 3. Build a local proof page - Use a same-lab auto-submitting form with a non-sensitive disposable password.

```
python3 -m http.server 8000 --directory ./csrf-proof
page: http://192.168.56.10:8000/csrf_attack.html
```

Step 4. Verify the result - Open the proof page in the already-authenticated test profile and verify the password change.

```
HTTP 200 from DVWA
verification login: success with new lab password
```

CSRF output

The local test account password changed after the browser visited the proof page, demonstrating that the application accepted a state-changing request without a CSRF token.

Evidence	Observed result
Request protection	No CSRF token in the submitted form
Origin	Local host-only training page
Response	HTTP 200; action accepted
Verification	Disposable lab account accepted the new password
Mitigation	CSRF tokens, SameSite cookies, Origin/Referer checks, reauthentication

Result

CSRF was reproduced safely in the local lab. A production application should protect every state-changing request and should not rely on cookies alone to prove user intent.

Safety and Submission Notes

- Use a disposable local account and password.
- Do not send the proof page to real users.
- Restore the lab password and delete the proof page after testing.

Viva Questions and Answers

CSRF vs XSS?

CSRF tricks an authenticated browser into sending a request; XSS executes attacker-controlled script in the target origin.

Why are cookies not needed by the attacker?

The victim browser automatically attaches them to the target request.

How does a CSRF token help?

The attacker cannot normally read the unpredictable token required by the server.

Does CSRF work when logged out?

Usually not for authenticated actions because no valid session cookie is present.

Two helpful headers?

SameSite cookie policy and validation of Origin/Referer headers.